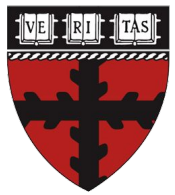
Lecture 7A: LLM: Agents II

AC215/E115

Pavlos Protopapas
Shivas Jayaram



Logistics / Reminders

1. Approx. **85%** of students have groups - **Milestone 1 due tonight 09/29 10 PM**

2. When you form a team, please fill this sheet too. https://bit.ly/ac215_2026_groups



Outline

1. Recap
2. Context Engineering
3. Agent Harness
4. Skills
5. Agent Systems
6. Evaluation
7. Additional Resources

Outline

1. **Recap**
2. Context Engineering
3. Agent Harness
4. Skills
5. Agent Systems
6. Evaluation
7. Additional Resources

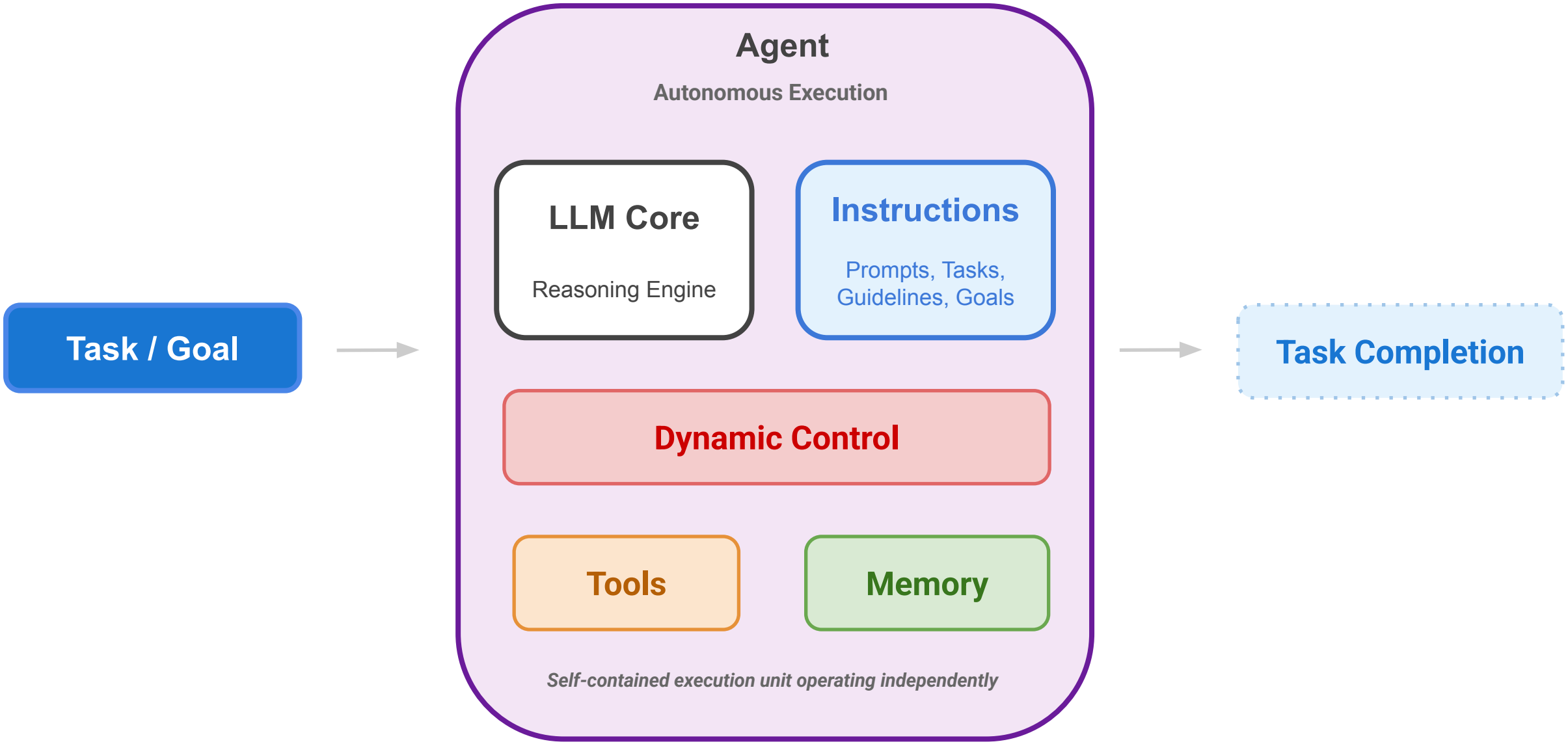
Recap: Agents

What is an Agent?

Recap: Agent Definition

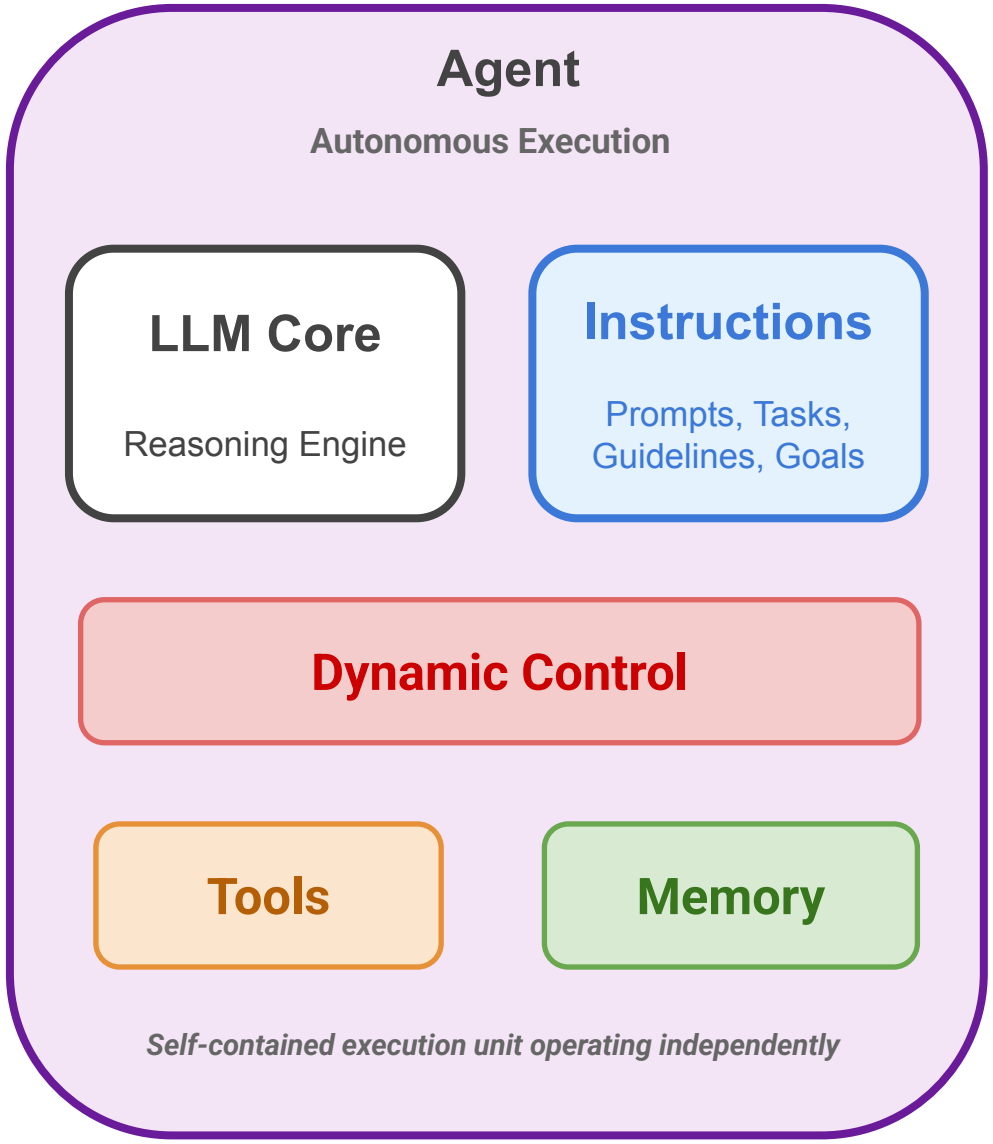
Agent is a system where an LLM, given instructions and tools, runs autonomously until a task is done.

Recap: What is an Agent?

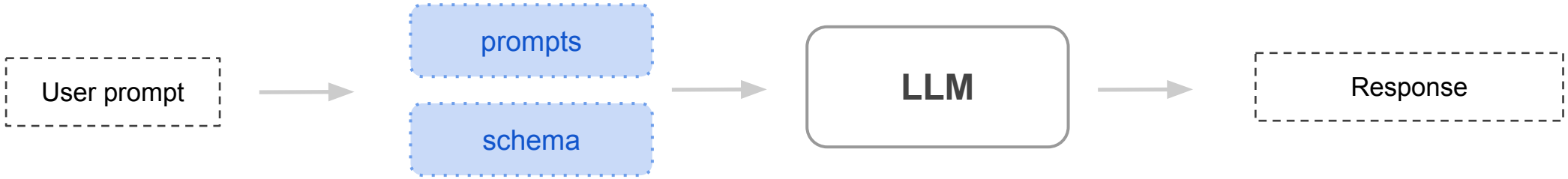


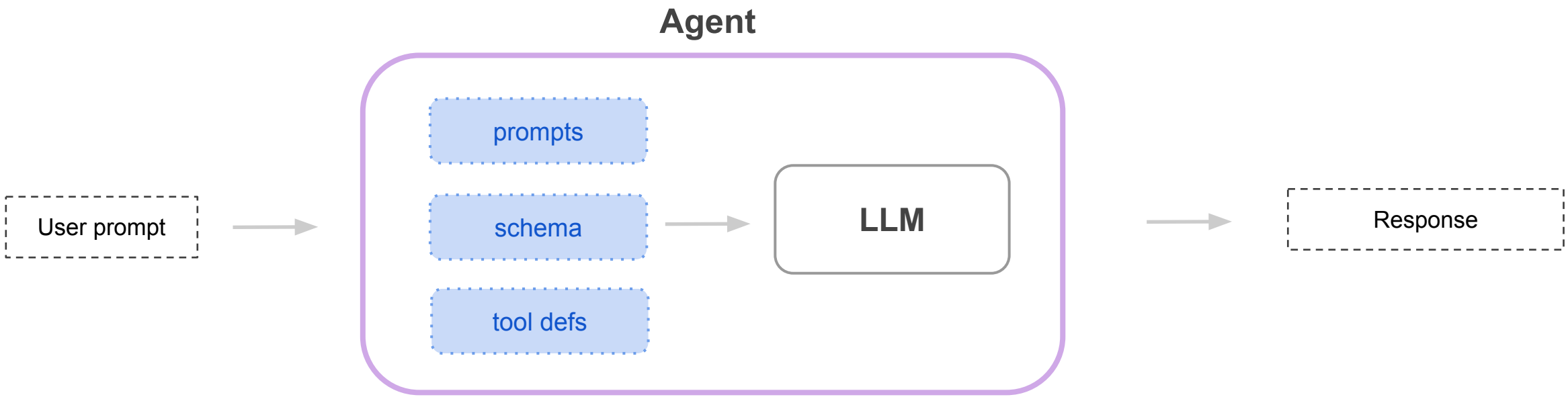
Recap: Agent Building Blocks

- **LLM Core**
- **Instructions**
  - System & User Prompts
  - Contexts
  - Output Schema
- **Tools**
  - Data
  - Actions
  - Orchestrations (flows)
- **Memory**
  - User sessions
  - User preferences

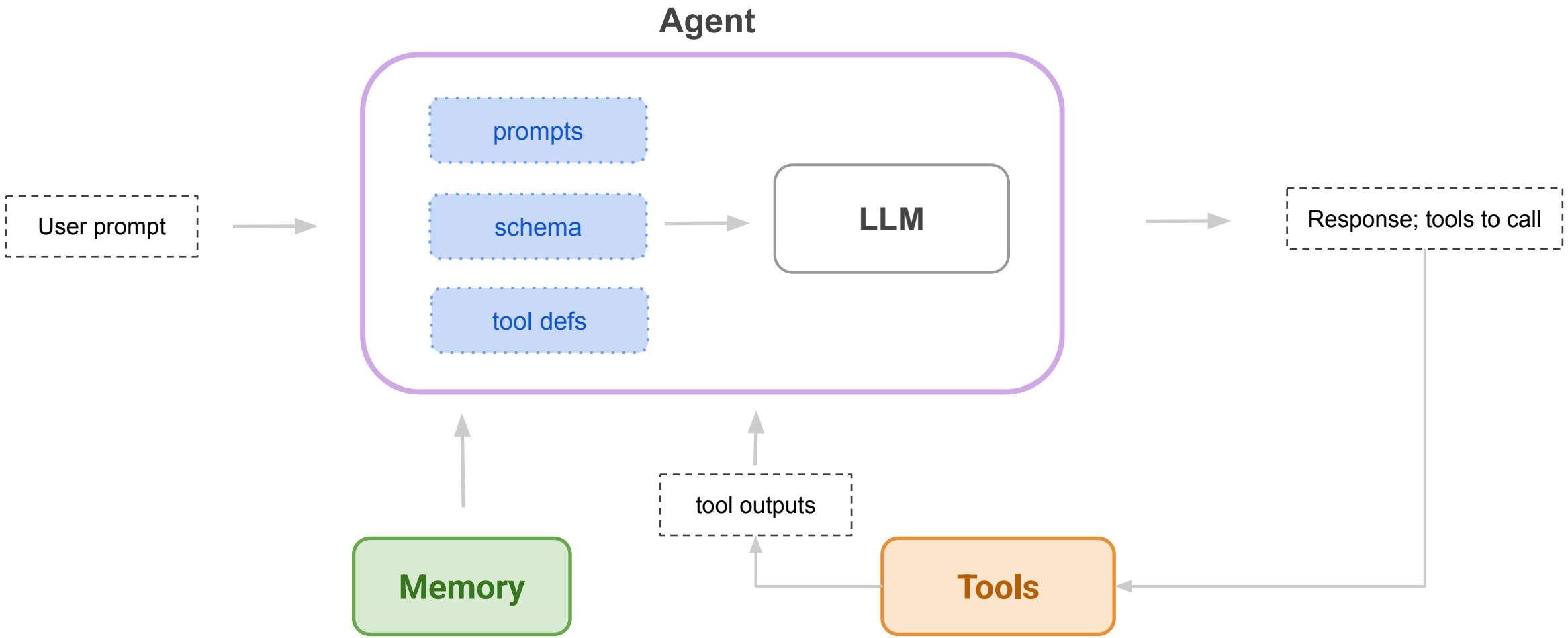


Motivation


User prompt
prompts
schema
LLM
Response


Agent
User prompt
prompts
schema
tool defs
LLM
Response

Motivation



How do we manage what goes into the LLM?

Outline

1. Recap
2. **Context Engineering**
3. Agent Harness
4. Skills
5. Agent Systems
6. Evaluation
7. Additional Resources

Context Engineering

What is Context Engineering?

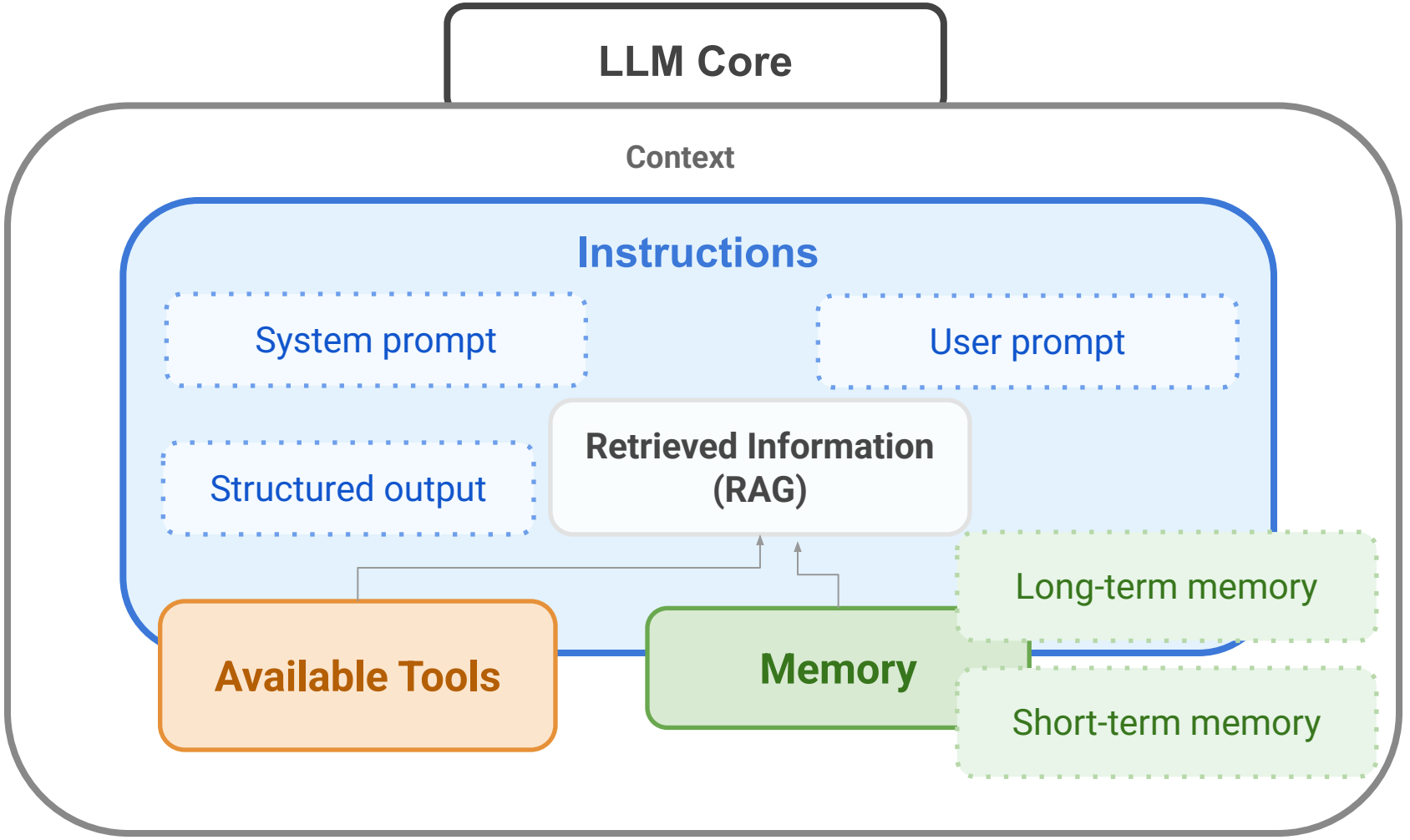
Think of context engineering as the invisible layer that feeds and connects all parts of the agent.

It manages what information the LLM sees, when it sees it, and how it uses it. Without context engineering, even the best reasoning model cannot plan, adapt, or remember effectively.

Instructions:

- Makes LLMs specialized with specific expertise.
- Uses the LLM to implement the decision maker and planner.
- Guides the planner to call tools.
- Retrieves memory to keep better context.

Context Engineering



Context Engineering

User Input

```
What's the best cheese for a grilled cheese sandwich?
```

Prompt Engineering

```
You are a culinary expert specializing in cheese and comfort foods.
When recommending cheeses, consider: melting properties, flavor
profiles,
availability, and cooking techniques. Be specific and practical.
```

Memory (Current Conversation)

```
Previous context from this conversation:
- User mentioned they're cooking for kids (prefers mild flavors)
- User has a basic stovetop setup
- User asked about butter vs mayo for spreading
- Tool called: temperature_guide() returned optimal melting temps
```

RAG (External Data)

```
Based on the Cheese Institute's melting guide and recent taste test
data:
- American cheese: melts at 150°F, mild flavor
- Gruyère: melts at 170°F, nutty complex flavor
- Cheddar: varies by age, sharp varieties may separate
```

Memory (Past Conversation)

```
User preference data from previous conversations:
- Preferred mild cheddar in mac and cheese discussion (2 weeks ago)
- Mentioned lactose sensitivity when discussing pizza (1 month ago)
- Usually shops at Kroger (limited specialty cheese selection)
- Budget-conscious, typically chooses items under $6
```

Context Engineering

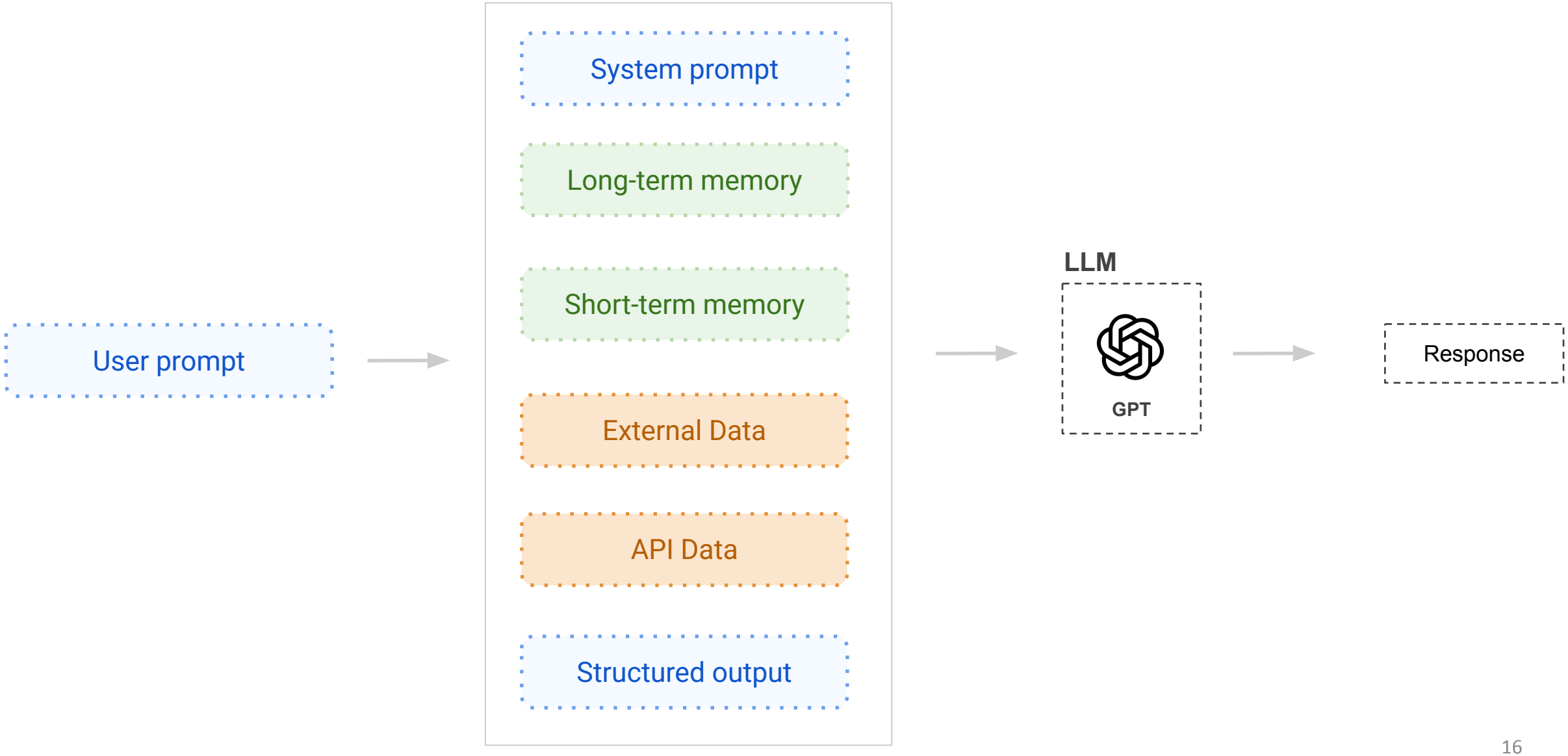
Tool Call Output

```
Results from function calls made during this conversation:
grocery_store_api("Kroger", "cheese", user_zipcode="29550"):
- Kraft American Singles: $4.99, in stock
- Kroger Brand Mild Cheddar: $3.49, in stock
- Tillamook Medium Cheddar: $5.99, in stock
- Boar's Head Vermont Cheddar: $8.99, in stock
nutrition_checker("lactose content", ["american", "cheddar"]):
- American cheese: 0.1g lactose per slice (processed, lactose removed)
- Mild cheddar: 0.2g lactose per slice (naturally low)
- Aged cheddar: 0.1g lactose per slice (aging reduces lactose)
melting_simulator("grilled_cheese", temperature=300, cheeses=["american", "mild_cheddar"]):
- American: smooth melt, no separation, kid-friendly texture
- Mild cheddar: good melt, slight oil separation at high heat, richer flavor
```

Structured Output

```
Respond in this JSON format:
{
 "top_recommendation": "cheese name and brief reason",
 "alternatives": ["option 1", "option 2"],
 "cooking_tips": ["tip 1", "tip 2"],
 "why_this_works": "explanation considering user's constraints"
}
```

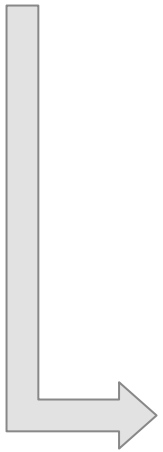
Context Engineering



Context Engineering

User Input

```
What's the best cheese for a grilled cheese sandwich?
```



Output

```
{
 "top_recommendation": "Kroger Brand Mild Cheddar ($3.49) - Perfect melting properties for stovetop cooking, works with your lactose sensitivity, and your kids already loved it in mac and cheese",
  "alternatives": [
   "Kraft American Singles ($4.99) - Smoothest melt, zero lactose, most kid-friendly",
   "50/50 blend of Mild Cheddar + American - Best of both worlds: flavor + perfect melt"
 ],
  "cooking_tips": [
   "Keep heat at medium-low (around 300°F) to prevent the cheddar from separating",
   "Use mayo on the outside of bread …",
 ],
  "why_this_works": "Based on your Kroger shopping habits, this is the most budget-friendly option at $3.49. The mild cheddar has minimal lactose (0.2g per slice)…"
}
```

Context Engineering

- **Context** = everything supplied to the LLM call: instructions, user input, history, retrieved evidence, tool definitions, tool results
- **Prompt Engineering** = the instructions and examples
- **Context Engineering** = the whole information lifecycle: what to include, when, and how to maintain it over turns
- **Who does it?** the app / agent developer — every block your code chooses to include is a decision

Outline

1. Recap
2. Context Engineering
3. **Agent Harness**
4. Skills
5. Agent Systems
6. Evaluation
7. Additional Resources

What Is an Agent Harness

- "*The software scaffolding around a model: the loop, tools, context management, and guardrails that turn raw intelligence into a working agent*" — Anthropic
- The model produces text or proposes an action. It does not and cannot execute the tool function — the runtime does
- A harness is the runtime — Claude Code, Codex, Antigravity are all harnesses

One Turn

Think → Reason → Plan → Execute → Verify → Response

LLM

- **Think** — deliberation before committing *(the grey streamed text in Claude Code)*
- **Reason** — structured thinking: hypotheses, alternatives, consistency
- **Plan** — which actions, in what order

Harness

- **Execute** — the harness dispatches: tools, code, sub-agents
- **Verify** — check the work before trusting it: code checks, judges
- **Response** — the harness decides and assembles what to show the user. LLM output, visible reasoning, tool call details, rich UI/UX components

Action Boundary

LLM probabilistic · adaptive · open-ended	*who decides?*	**Harness** code · deterministic · auditable
The plan · which step comes next · when the task is done	**Control flow**	The loop itself · max turns · timeouts · stop conditions
Which tool · what arguments · how to react to an error	**Tools**	Tool registry · schemas · argument validation · tool execution · errors
What to retrieve · which skill to load · what is worth remembering	**Context & memory**	System prompt · instruction vs evidence lanes · compaction trigger · what persists and where
Self-critique · revise using the checker's feedback	**Verification**	Schema + fact checks · tests · deterministic gates
When to delegate · how to brief the worker · how to merge results	**Sub agents**	Spawn limits · context isolation · concurrency · per-agent tool scopes

Move LEFT: goals are novel · model is capable · quicker time to market · exploring is worth the tokens

Move RIGHT: stakes are high · actions irreversible · task predictable · tokens matter · audit required

Harness Engineering

Set boundaries carefully — permissions · sandboxing · budgets · checks

LLM
probabilistic · adaptive · open-ended

who chooses the next step?

Harness
code · deterministic · auditable

Move to LLM ← What can the harness stop doing?
model got more capable · goals turned out varied · code is brittle to maintain

Move to Harness → What can the model stop doing?
same decision keeps repeating · output is predictable · tokens, latency or audit matter

The harness keeps **authority** at every point of the conversation

Who does Harness Engineering?

- Platform / framework engineers: the people behind Claude Code, Codex, the Agents SDK
- **You**, if you've written `while True: model(); dispatch()`

Outline

1. Recap
2. Context Engineering
3. Agent Harness
4. **Skills**
5. Agent Systems
6. Evaluation
7. Additional Resources

Skills

- Skills are "*Organized folders of instructions, scripts, and resources that agents can discover and load dynamically*" — Anthropic
- Is a folder with **SKILL.md** + optional references, templates, scripts
- It packages procedural guidance, examples, references, or scripts
- Solves two problems: the **expertise gap** (your process, your style, your rules) and the **context budget** (loads only when relevant)
- Domain experts & developers can contribute to a skill

Anatomy of a Skill

```
cheese-pairing/
├── SKILL.md                      ← required: frontmatter + instructions
├── references/pairing_table.md   ← docs, loaded on demand
└── scripts/lookup.py             ← code the agent RUNS

---
name: cheese-pairing
description: Expert cheese pairing recommendations for wines, beers, and
  accompaniments. Use when the user asks what to serve, pour, or eat with a cheese.
---
1. Identify each cheese.
2. Run scripts/lookup.py <cheese> for the house pairing.
3. Report one safe pairing, one adventurous pairing, one serving tip.
```

- `name` must match the folder; the **description** is what the agent sees before deciding to load — a **hint to the model, not a deterministic trigger**

Progressive Disclosure

Level	What loads	When	Cost (authoring guidance)
1 **Discover**	name + description of every skill	startup	~100 tokens / skill
2 **Activate**	the selected SKILL.md	task matches	< 5k tokens recommended
3 **Use resources**	references, script results	as needed	only what's read or returned

🧀 This is context engineering, automated — and the harness executes it. The two sections we saw, combined.

Tools, MCP, & Skills

	Tool	MCP	Skill
Question	What operation can be requested?	How are capabilities and data exposed to a client?	What procedure should guide this task?
Grants	an **action**	**connectivity**	**know-how**
Analogy	the hands	the universal socket	the training manual
Authored by	software engineer	platform / integration engineer	**domain expert**

🧀 MCP gives the agent reach; skills give it guidance; tools are the hands.

Outline

1. Recap
2. Context Engineering
3. Agent Harness
4. Skills
5. **Agent Systems**
6. Evaluation
7. Additional Resources

Agent Systems

Single Agent System

Consist of a single agent that execute flows in loops

Multiple Agent System

Consist workflow executions distributed across multiple agents

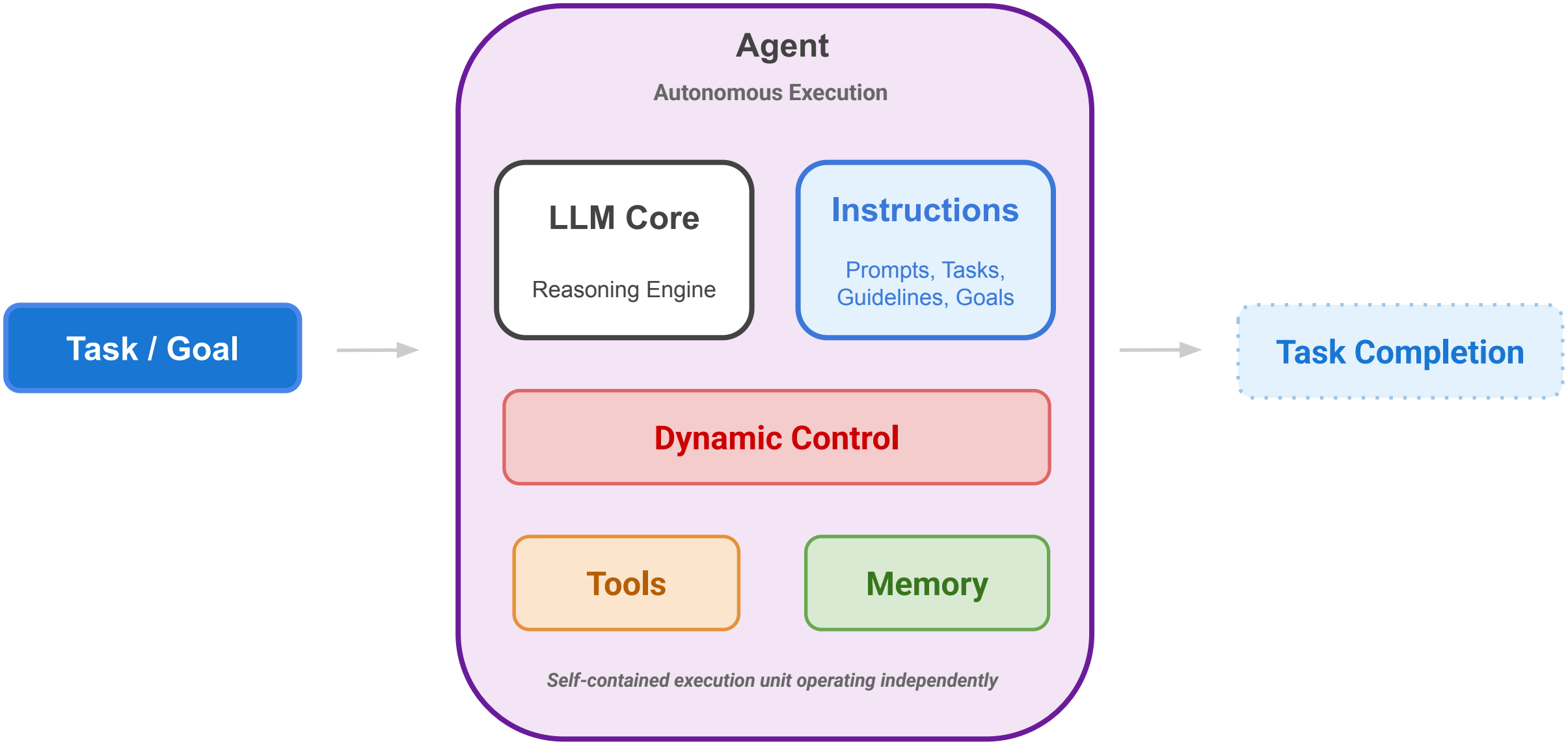
Single Agent Systems

- Consists of a single LLM with instructions + tools
- Handle multiple tasks by adding tools incrementally
- Agent complexity is managed by controlling number of tools
- Evaluation and maintenance is manageable

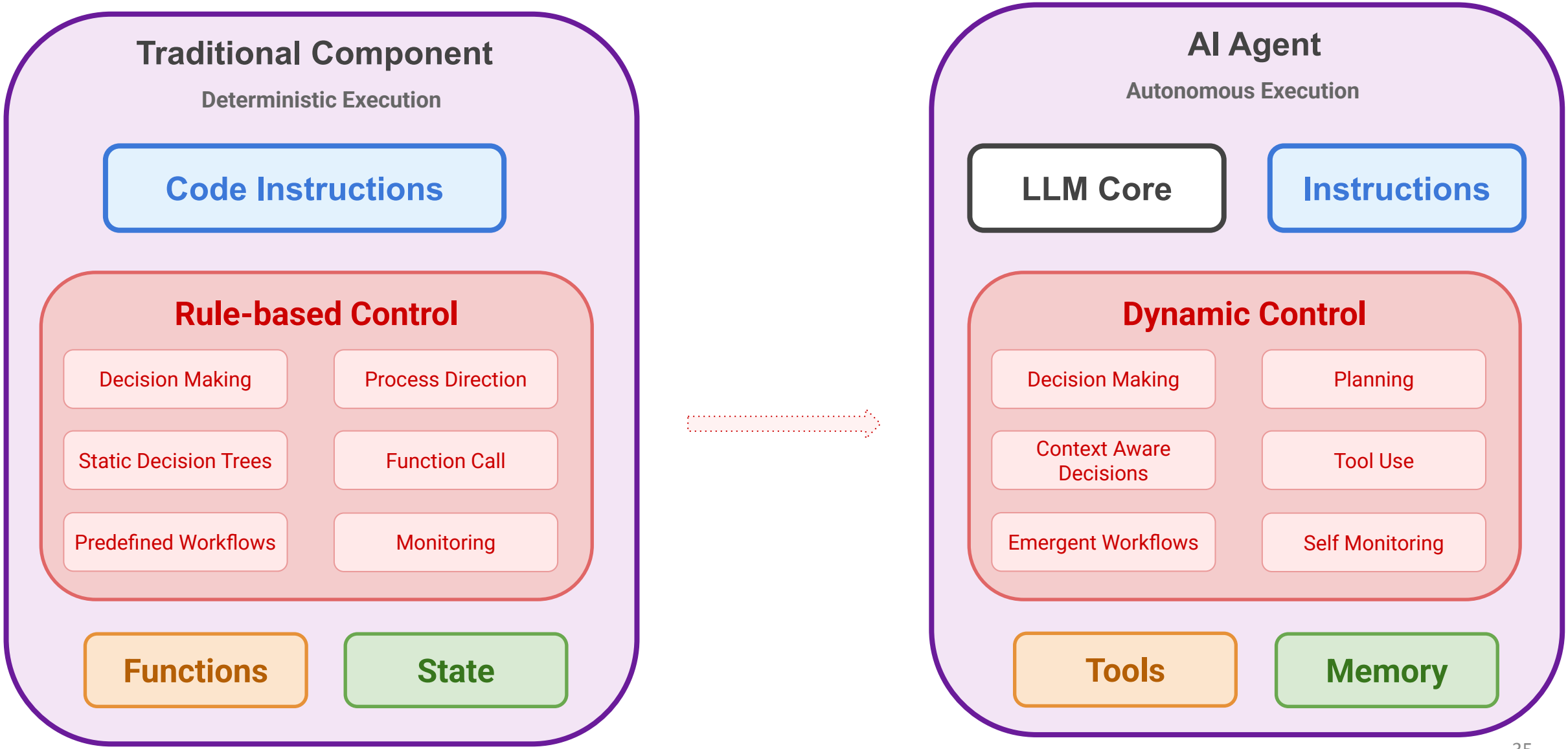
Multi Agent Systems

- Consists of a workflow across multiple agents
- Multi-agents can be modeled as graphs, with nodes as agents
- Manager Orchestration Pattern (Agents as tools)
  - A manager agent coordinates multiple agents using tool calls
  - Each agent handles very specialized tasks
  - Edges in the graph represent tool calls
- Decentralized Orchestration Pattern (Agent 2 Agent)
  - Agents operate as peers handing over tasks to one another based on specialization
  - Edges in the graph represent handoffs

Review: Agent



From Traditional to AI Agent Systems



The Autonomy Spectrum

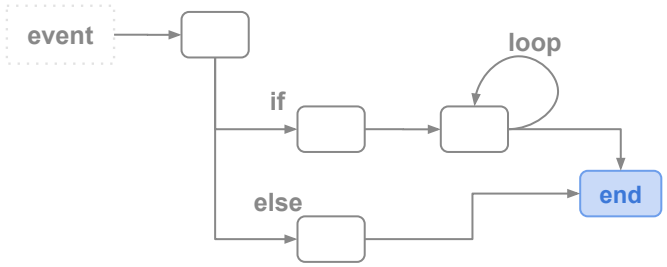
Traditional Component

Deterministic Execution

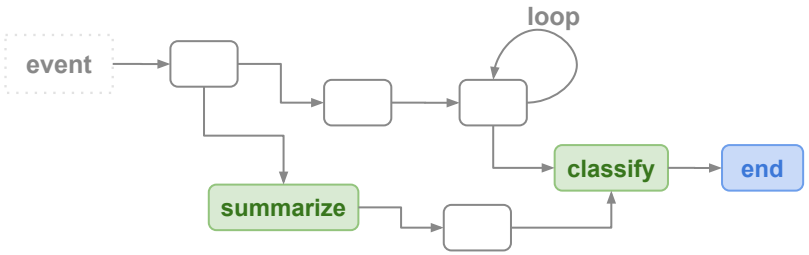
AI Agent

Autonomous Execution

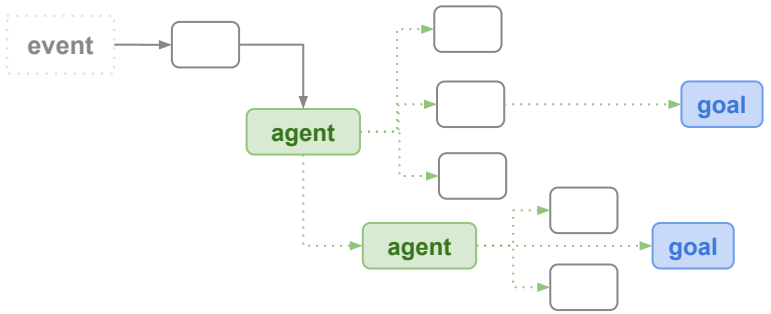
Directed Graphs



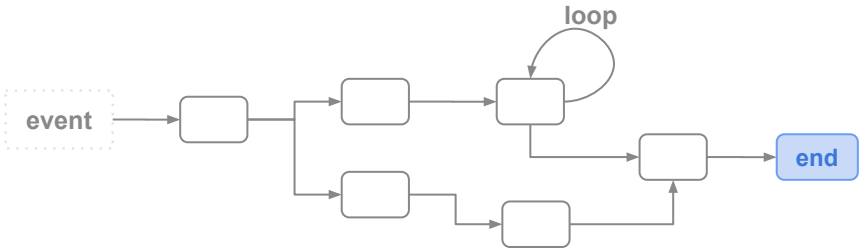
Workflow Orchestrators (with ML)



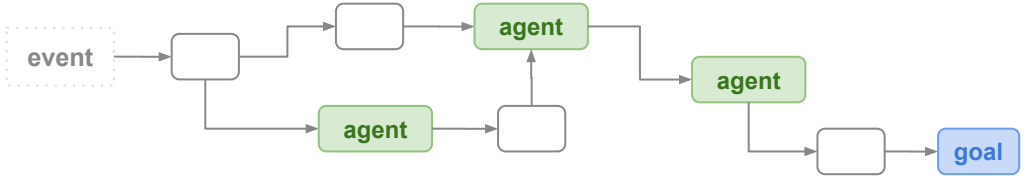
Autonomous Orchestrators



Workflow Orchestrators



Hybrid Orchestrators



Rule-based Systems

Predictable

Data Driven Workflows

ML-Enhanced Tasks

LLM-Enhanced Tasks

Structured Output

Guided / Bounded Tool Selection

Full Agent Autonomy

Agent Layers

Layer	Controls	Primary owner
LLM	What gets generated from the context it's given: the reasoning, the plan, the answer	LLM provider
Tool	What the agent can actually do in its environment: each capability's contract, inputs, outputs, side effects	Software Engineer
Context	What the model receives next: instructions, evidence, memory, tool results	AI Engineer
Agent Harness	How execution proceeds and what is permitted: the loop, tool dispatch, limits, checks	Platform / AI Engineer
Skills	Which reusable procedure the agent loads on demand	Domain expert

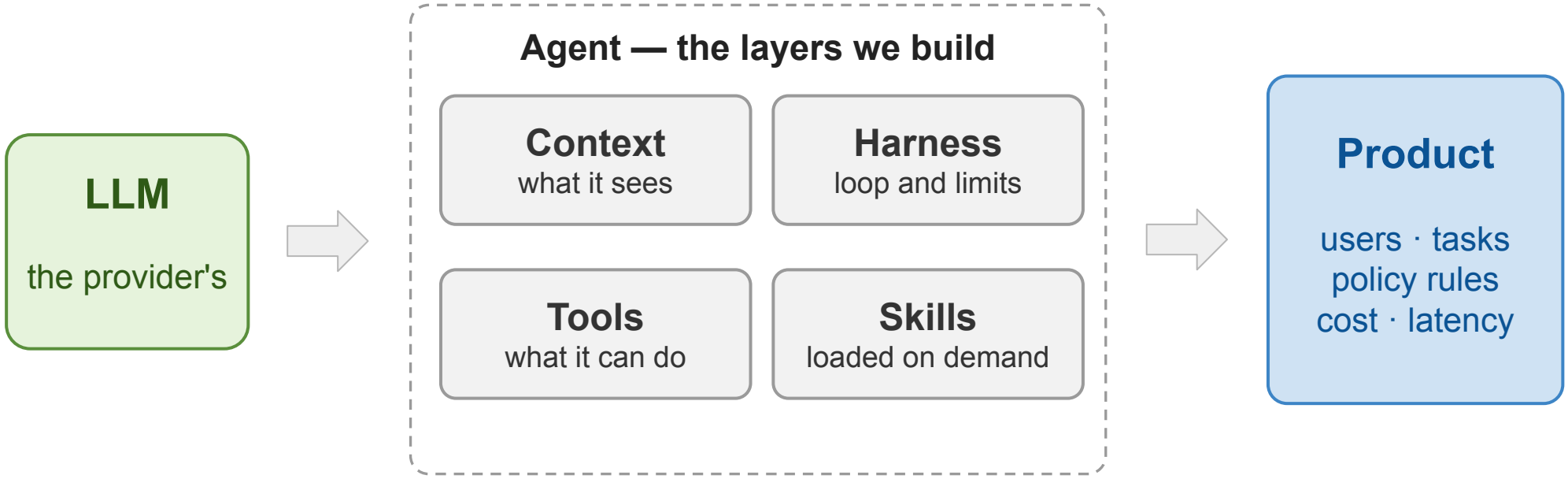


For this class and small teams you will wear all hats

Outline

1. Recap
2. Context Engineering
3. Skills
4. Agent Harness
5. Agent Systems
6. **Evaluation**
7. Additional Resources

From Model to Product



Every layer is something we change.
How do we test and evaluate those changes?

What to Measure?

Outcome
did it get the right result?

Trajectory
did it get there sensibly?

Cost
tokens · turns · latency

Layer	What a change here typically breaks	What catches it
LLM	Swapped model: answers drift, output format changes	Outcome
Tool	Bad arguments, unexpected side effects	Trajectory
Context	Prompt edit fixes one case, breaks three others	Outcome
Agent harness	Loop runs long, a gate gets skipped	Trajectory, Cost
Skills	Wrong skill loaded, steps skipped	Trajectory

How to evaluate?

Deterministic checks
tests · schema · state assertions

cheap · rigid · auditable

LLM as judge
rubric · k samples

flexible · needs calibration

Human review
spot-check the judge

slow · costly · gold standard



who decides pass / fail?

Move as far left as the task allows — same rule as the harness

Evaluation Process

1. Failure observed
a real user or run broke

2. Capture test case
input · expected · limits

3. Change one layer
prompt · tool · harness · skill

4. Run the suite k times
pass rate and variance

5. Compare to baseline
a diff, not a green check

6. Merge or iterate
traces kept either way

The suite grows over time; the eval set is the most valuable artifact

Outline

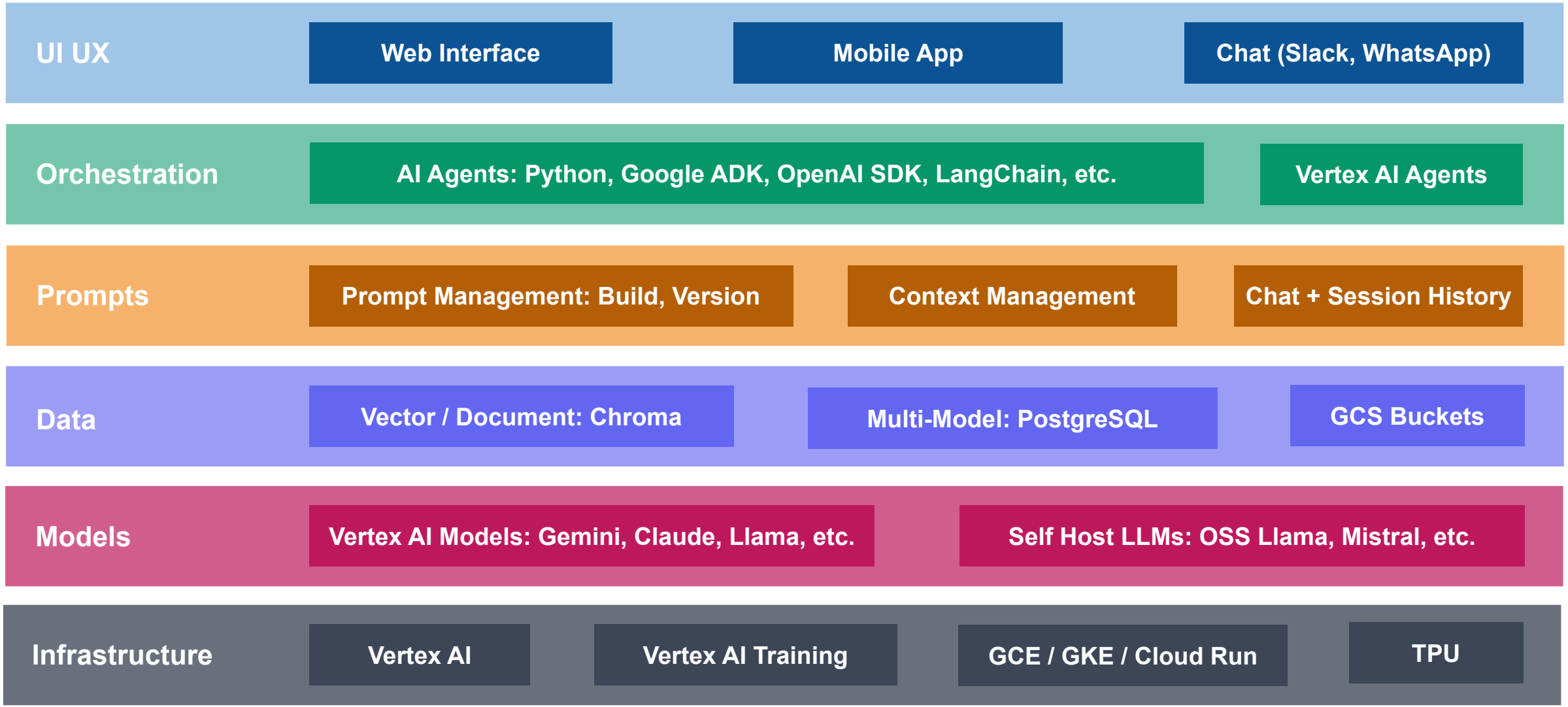
1. Recap
2. Context Engineering
3. Skills
4. Agent Harness
5. Agent Systems
6. Evaluation
7. **Additional Resources**

Tutorial: Agent Harness

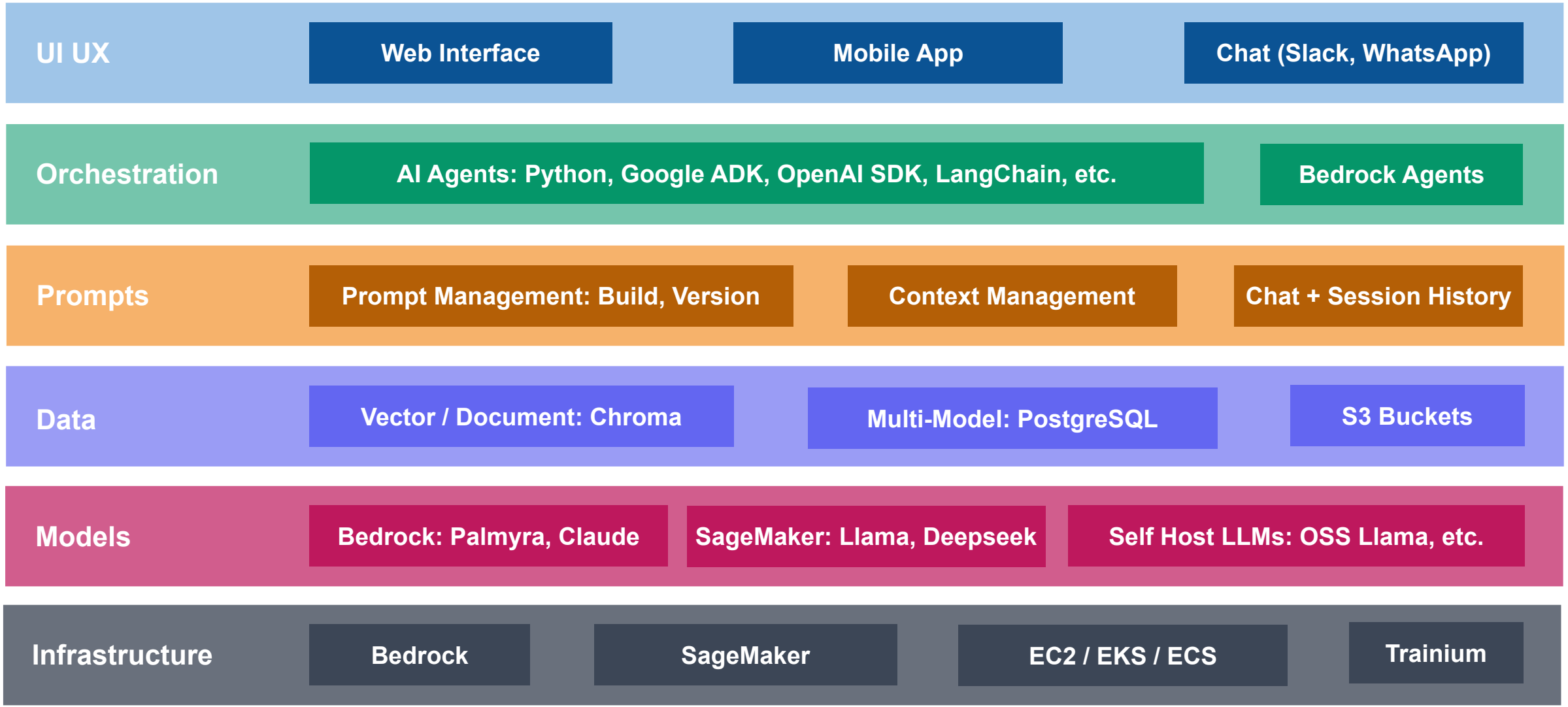
Tutorials

- Notebook: https://colab.research.google.com/drive/1ZYMsbG0gqsbs-6Pcj8WtT3WhEa4f80EZ?usp=sharing
- Github: https://github.com/dlops-io/agent-harness

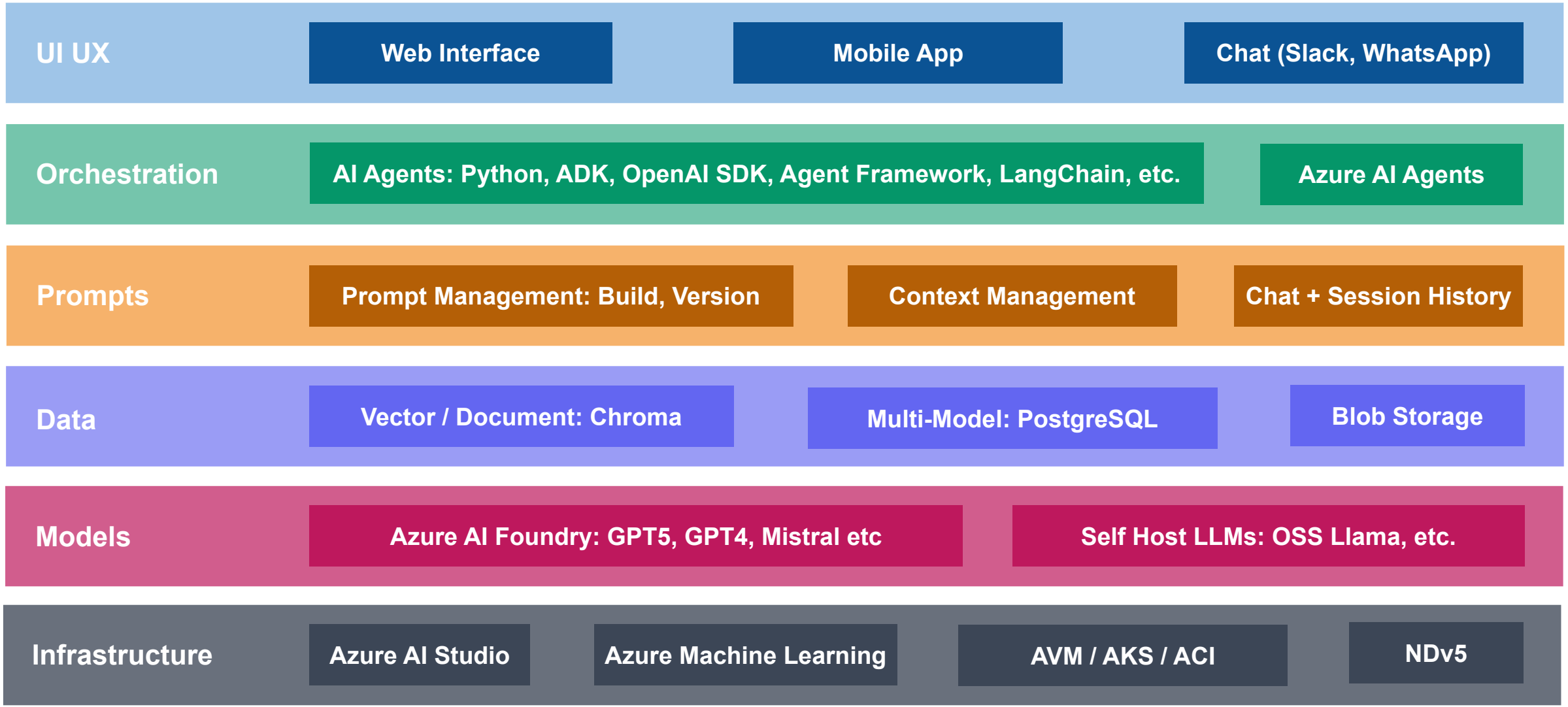
GenAI Building Blocks (GCP)



GenAI Building Blocks (AWS)



GenAI Building Blocks (Azure)



GenAI Frameworks [TODO]

Data & Embeddings

- Chroma
- Pinecone
- Milvus
- Weaviate
- Qdrant

- Elasticsearch
- PostgreSQL
- MongoDB
- Neo4j
- Oracle

- Vector Search (GCP)
- OpenSearch (AWS)
- Azure AI Search (Azure)

Prompts

- Agenta
- Maxim AI
- LangSmith
- PromptLayer
- DSPy

- Vertex AI Prompt Gallery (GCP)
- Bedrock Prompt Management (AWS)
- PromptFlow (Azure)

Orchestration

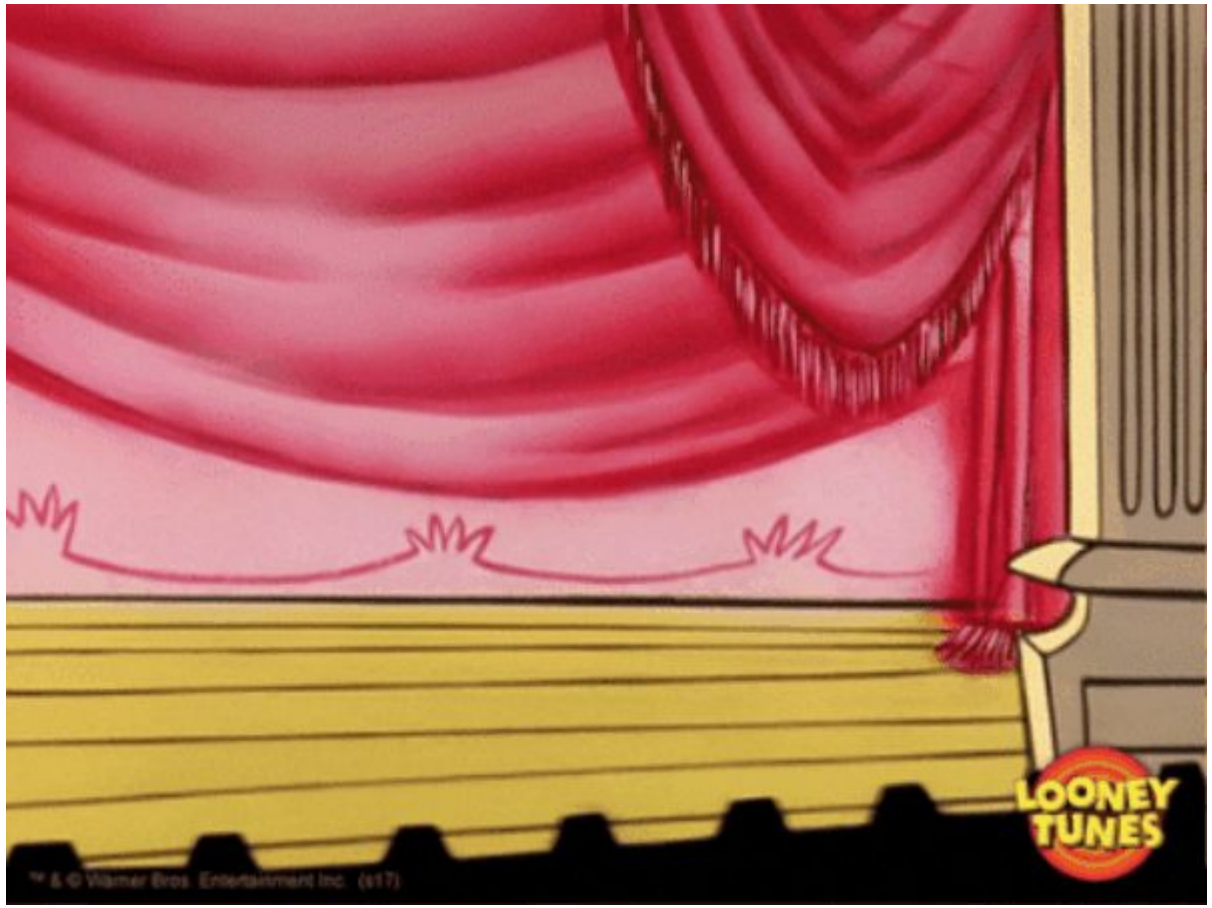
- OpenAI Agent SDK
- Google ADK
- LangChain
- LangGraph
- LlamaIndex
- Semantic Kernel

- Vertex AI Agents (GCP)
- Bedrock Agents (AWS)
- Azure AI Agents (Azure)

Low-code Orchestration

- N8N
- Crew AI
- Flowise (Workday)
- ServiceNow AI Agents
- Moveworks Agents
- Oracle AI Agents

- Vertex AI Agents (GCP)
- Bedrock Agents (AWS)
- Azure AI Agents (Azure)



THANK YOU